서브그래프 학습 가이드

TokenZswap의 이벤트를 The Graph로 인덱싱하기
스키마 · 매니페스트 · 매핑 · 배포 · 검증

대상 폴더: `code/subgraph/` (schema.graphql, subgraph.yaml, src/amm.ts, src/nft.ts)
도구: graph-cli 0.98.1, graph-ts 0.38.2, Subgraph Studio, 네트워크 `chapel` (BSC 테스트넷)
이 문서의 쿼리 결과와 체인 값은 실제 배포한 서브그래프와 BSC 테스트넷에서 확인한 값입니다.

목차

1. 서브그래프는 왜 필요한가
2. The Graph의 구조
3. tx 하나가 장부가 되기까지
4. 프로젝트 준비: 폴더, ABI, codegen
5. schema.graphql: 장부 설계
6. subgraph.yaml: 무엇을 볼지
7. startBlock: 어디서부터 읽을지
8. 매핑 코드 (1): AssemblyScript와 생성된 클래스
9. 매핑 코드 (2): 핸들러 한 줄씩
10. 빌드와 배포
11. 검증: 서브그래프 vs 체인
12. GraphQL 쿼리 문법
13. 우리가 겪은 에러 모음
14. 한계와 주의점
15. 스스로 점검하기
16. 부록: 명령어와 엔티티 한 장 요약

읽는 법

ERC-20 가이드와 NFT 가이드에서 본 **이벤트**(5장, 9장)가 이 문서의 출발점입니다. 서브그래프는 새 정보를 만들지 않습니다. **컨트랙트가 남긴 이벤트를 처음부터 다시 읽어 장부를 만드는 것**이 전부입니다. 이 한 문장을 기억하고 읽으세요.

CHAPTER 1

서브그래프는 왜 필요한가

컨트랙트의 `view` 함수는 **지금 이 순간의 상태**만 알려 줍니다. "`reserveA`는 얼마?"에는 답하지만, "**지금까지** 누가 swap했나?"나 "**판매 중인 것만** 보여 줘"에는 답하지 못합니다.

1-1. 프론트에서 실제로 겪은 불편함

화면	지금 방식	문제
내 NFT	`ownerOf(0)` ~ `ownerOf(19)` 20번 호출	20번 이상은 안 보임. `_nextTokenId`가 private이라 총 개수를 모름
마켓	`listings(0)` ~ `listings(n-1)`을 전부 읽고 active만 거름	건수가 늘수록 느려짐
거래 내역	불가능	storage는 "지금"만 보관하고 "역사"는 남기지 않음

1-2. 역사는 이벤트에 있다

역사는 **이벤트 로그**에 남아 있습니다. 모든 swap은 `Swap(trader, tokenIn, amountIn, amountOut)` 로그를 남겼습니다. 그런데 RPC로 로그를 직접 뒤지는 방식(`eth_getLogs`)에는 문제가 있습니다.

- 한 번에 조회할 수 있는 **블록 범위가 제한**됩니다(노드마다 다름). 배포 이후 전체를 보려면 여러 번 나눠 물어야 합니다.
- 로그는 "일어난 일"의 목록일 뿐입니다. "지금 판매 중인 것"을 알려면 등록, 취소, 판매 로그를 **직접 조합**해야 합니다.
- 페이지를 열 때마다 이 작업을 다시 해야 합니다.

> **서브그래프의 역할**
>
> 이벤트를 **한 번만** 처음부터 읽으면서, 우리가 정한 규칙(매핑 코드)대로 **정리된 장부(DB)**를 만들어 두는 서비스입니다. 프론트는 블록체인을 뒤지는 대신, 그 장부에 **GraphQL로 질문**만 하면 됩니다.

비유하자면 블록체인은 **영수증이 시간순으로 쌓인 상자**이고, 서브그래프는 그 영수증을 처음부터 읽으며 장부를 정리해 두는 **회계 담당자**입니다. 새 영수증(새 블록)이 들어올 때마다 장부도 바로 고칩니다.

질문	서브그래프 쿼리
이 주소가 가진 NFT는?	`nfts(where: { owner: "0x…" }) { id }`
판매 중인 건은?	`listings(where: { status: ACTIVE }) { id price }`
최근 swap 10개는?	`swaps(first: 10, orderBy: timestamp, orderDirection: desc) { … }`

CHAPTER 2

The Graph의 구조

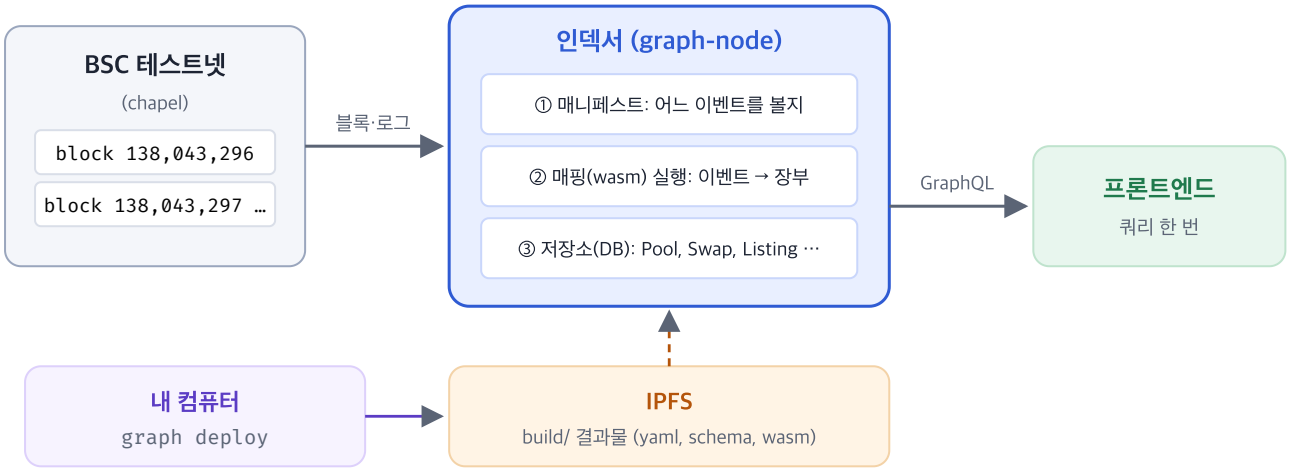


그림 1. 배포하면 build 결과물이 IPFS에 올라가고, 인덱서가 그걸 받아 체인을 읽으며 장부를 만듭니다.

구성 요소	역할
graph-node (인덱서)	블록을 순서대로 읽고, 매니페스트에 적힌 이벤트를 찾으면 매핑 함수를 실행해 장부를 고침
매니페스트 (`subgraph.yaml`)	어느 체인, 어느 컨트랙트, 어느 이벤트, 몇 번 블록부터
스키마 (`schema.graphql`)	장부(테이블)의 모양. 쿼리할 수 있는 모양이기도 함
매핑 (`src/*.ts` → wasm)	"이 이벤트가 오면 장부를 이렇게 고쳐라"는 코드. AssemblyScript로 쓰고 WebAssembly로 컴파일
Subgraph Studio	The Graph의 개발자 화면. 서브그래프를 만들고, 배포하고, Playground에서 쿼리하고, 동기화 상태를 봄

네트워크 이름

The Graph는 체인마다 정해진 이름을 씁니다. BSC 테스트넷은 **`chapel`**, BSC 메인넷은 `bsc`, Sepolia는 `sepolia` 입니다. The Graph 공식 네트워크 레지스트리(2026-10-08 갱신)에서 `chapel`의 Studio 지원을 확인했습니다.

CHAPTER 3

tx 하나가 장부가 되기까지

실제로 BSC에서 실행한 **"NFT #0을 20 SW42에 판매 등록"** tx 하나를 따라가 봅니다. 이 tx는 로그를 2개 남깁니다.



그림 2. 데이터 소스가 다른 두 로그가 각자의 핸들러를 실행하고, 각자의 장부를 고칩니다.

순서	일어나는 일
1	인덱서가 새 블록을 받음
2	블록 안의 로그 중 **주소가 TokenizeArt이고 topic0이 Transfer**인 것 발견 → `handleTransfer` 실행
3	**주소가 AMM이고 topic0이 NFTListed**인 로그 발견 → `handleNFTListed` 실행
4	각 핸들러가 `.save()`한 엔티티가 저장소에 기록됨
5	몇 초 뒤 GraphQL 쿼리에 바로 반영 (실제로 확인)

같은 블록 안에서는 **로그 순서대로** 핸들러가 실행됩니다. 그래서 swap-trace 문서에서 본 로그 순서(Transfer → Transfer → Swap)가 그대로 처리 순서가 됩니다.

CHAPTER 4

프로젝트 준비: 폴더, ABI, codegen

```
code/subgraph/
├── package.json          graph-cli 0.98.1, graph-ts 0.38.2
├── .gitignore            node_modules/, generated/, build/
├── schema.graphql        장부 설계 (5장)
├── subgraph.yaml         매니페스트 (6장)
├── abis/
│   ├── AMM.json          forge 빌드 결과(out/)에서 "abi" 부분만 추출
│   └── TokenizeArt.json
├── src/
│   ├── amm.ts            AMM 이벤트 핸들러 6개
│   └── nft.ts            NFT Transfer 핸들러 1개
├── generated/            graph codegen이 만듦 (커밋 안 함)
└── build/                graph build가 만듦 (커밋 안 함)
```

폴더를 `code/subgraph/`에 둔 이유는 과제가 요구하는 최상위 구조(`README.md`, `code/`, `frontend/`, `deployment/`, `documentation/`)를 지키기 위해서입니다.

4-1. ABI는 어디서?

Foundry가 컴파일하면 `code/out/AMM.sol/AMM.json`에 바이트코드, 메타데이터, ABI가 함께 들어 있습니다. 서브그래프는 **ABI만** 필요해서 그 부분만 꺼내 `abis/`에 저장했습니다.

> **컨트랙트를 고치면 ABI도 다시**
>
> 이벤트를 바꾸고 재배포했다면 `abis/`도 다시 추출해야 합니다. 그렇지 않으면 매니페스트의 시그니처와 ABI가 어긋나 codegen에서 에러가 납니다.

4-2. `graph codegen`: ABI와 스키마로 TypeScript 클래스 만들기

생성 파일	무엇에서	들어 있는 것
`generated/AMM/AMM.ts`	AMM ABI	이벤트 클래스(`Swap`, `NFTListed` …)와 `event.params.trader` 같은 getter, 컨트랙트 바인딩 `AMM.bind(주소)`
`generated/TokenizeArt/TokenizeArt.ts`	NFT ABI	`Transfer` 이벤트 클래스 등
`generated/schema.ts`	schema.graphql	엔티티 클래스 `Pool`, `Swap` …: `load`, `new`, `save`, 각 칸의 getter/setter

예를 들어 `generated/AMM/AMM.ts`에서 `class Swap extends ethereum.Event`를 찾으면 `get trader(): Address`가 있습니다. **ABI가 코드가 된 것**입니다. 그래서 매핑에서 `event.params.trader`를 쓰면 편집기가 타입까지 알려 줍니다.

CHAPTER 5

schema.graphql: 장부 설계

스키마는 **"무슨 질문에 답하고 싶은가"에서 거꾸로** 정합니다.

질문	엔티티	바뀌나?
지금 풀의 A, B, LP 총량은?	`Pool` (1줄)	계속 바뀜
swap 기록은?	`Swap` (swap마다 1줄)	안 바뀜
공급/회수 기록은?	`LiquidityEvent`	안 바뀜
판매 중인 것은?	`Listing` (판매 건마다)	상태가 바뀜
이 주소의 NFT는?	`Nft` (번호마다)	주인이 바뀜

```
type Pool @entity(immutable: false) {
  id: Bytes!                # AMM 주소
  tokenA: Bytes!            # Swap의 tokenIn이 A인지 B인지 구분용
  tokenB: Bytes!
  reserveA: BigInt!
  reserveB: BigInt!
  totalLiquidity: BigInt!   # LP 총량
  swapCount: BigInt!
}

type Swap @entity(immutable: true) {
  id: Bytes!                # tx 해시 + 로그 번호
  trader: Bytes!
  tokenIn: Bytes!
  amountIn: BigInt!
  amountOut: BigInt!
  timestamp: BigInt!
  txHash: Bytes!
}

enum LiquidityType { ADD REMOVE }
type LiquidityEvent @entity(immutable: true) {
  id: Bytes!  type: LiquidityType!  provider: Bytes!
  amountA: BigInt!  amountB: BigInt!  liquidity: BigInt!  timestamp: BigInt!
}

enum ListingStatus { ACTIVE SOLD CANCELLED }
type Listing @entity(immutable: false) {
  id: ID!                   # listingId ("0", "1", …)
  seller: Bytes!  nft: Bytes!  tokenId: BigInt!  paymentToken: Bytes!  price: BigInt!
  status: ListingStatus!
  buyer: Bytes              # 팔린 건에만 있음 → ! 없음
  createdAt: BigInt!
}

type Nft @entity(immutable: false) {
  id: ID!                   # tokenId ("0", "1", "2")
  owner: Bytes!
}
```

5-1. 문법

표기	뜻
`type X @entity(…)`	테이블 하나
`id`	모든 엔티티의 기본 키 (필수)
`!`	필수. 없으면 저장할 때 에러
`Bytes` / `BigInt` / `ID`	주소·해시 / uint256 같은 큰 정수 / 문자열 id
`enum`	정해진 값 중 하나. 매핑에서는 문자열 `'ACTIVE'`로 씀
`"""설명"""`, `#`	문서용 설명 / 주석

5-2. immutable: 기록 vs 상태

	`immutable: true`	**`immutable: false`**
뜻	한번 저장하면 수정 안 함	수정 가능
우리 엔티티	Swap, LiquidityEvent (이미 일어난 일)	Pool, Listing, Nft (상태)
장점	인덱싱이 빠르고 저장 공간이 적음	

5-3. id를 어떻게 정했나

엔티티	id	이유
Pool	AMM 주소	풀은 컨트랙트 하나 = 하나
Swap, LiquidityEvent	`tx 해시 + 로그 번호`	한 tx에 이벤트가 여러 개일 수 있음. 해시만 쓰면 겹칠 수 있어서 로그 번호를 붙여 유일하게
Listing	`listingId.toString()`	컨트랙트의 판매 번호와 1:1. 취소·판매 이벤트에서 `Listing.load("0")`으로 바로 찾음
Nft	`tokenId.toString()`	NFT 번호와 1:1

CHAPTER 6

subgraph.yaml: 무엇을 볼지

```
specVersion: 1.3.0
indexerHints:
  prune: auto                          # 오래된 과거 상태는 정리 (용량 절약)
schema:
  file: ./schema.graphql
dataSources:
  - kind: ethereum/contract
    name: AMM                          # generated/AMM/… 폴더 이름이 됨
    network: chapel                    # BSC 테스트넷
    source:
      address: "0x0a1134046cECA785a218A92376fdb542cB9ACA52"
      abi: AMM                         # 아래 abis의 name
      startBlock: 138043275
    mapping:
      kind: ethereum/events
      apiVersion: 0.0.9
      language: wasm/assemblyscript
      entities: [Pool, Swap, LiquidityEvent, Listing]
      abis:
        - name: AMM
          file: ./abis/AMM.json
      eventHandlers:
        - event: LiquidityAdded(indexed address,uint256,uint256,uint256)
          handler: handleLiquidityAdded
        - event: LiquidityRemoved(indexed address,uint256,uint256,uint256)
          handler: handleLiquidityRemoved
        - event: Swap(indexed address,indexed address,uint256,uint256)
          handler: handleSwap
        - event: NFTListed(indexed uint256,indexed address,indexed
address,uint256,address,uint256)
          handler: handleNFTListed
        - event: NFTListingCancelled(indexed uint256,indexed address)
          handler: handleNFTListingCancelled
        - event: NFTSold(indexed uint256,indexed address,indexed address,uint256)
          handler: handleNFTSold
      file: ./src/amm.ts
  - kind: ethereum/contract
    name: TokenizeArt
    network: chapel
    source:
      address: "0x7DA2B6BC5330ad801427E6879201b079A774aB15"
      abi: TokenizeArt
      startBlock: 138043275
    mapping:
      …
      entities: [Nft]
      eventHandlers:
        - event: Transfer(indexed address,indexed address,indexed uint256)
          handler: handleTransfer
      file: ./src/nft.ts
```

6-1. 데이터 소스 2개

NFT의 `Transfer`는 AMM이 아니라 **TokenizeArt 컨트랙트**가 냅니다. 그래서 지켜볼 컨트랙트가 두 개입니다. 한 서브그래프 안의 데이터 소스는 **모두 같은 네트워크**여야 합니다. 체인 하나를 처음부터 끝까지 읽는 구조이기 때문입니다.

6-2. 이벤트 시그니처 규칙

규칙	이유
Solidity 선언에서 **타입과 `indexed` 만** 남김 (이름 없음)	`event Swap(address indexed trader, …)` → `Swap(indexed address,…)`
공백 없음	`indexed`를 뺀 문자열의 keccak256이 topic0. 공백 하나로 다른 해시가 됨
`indexed`까지 ABI와 정확히 일치	디코더가 값을 topic에서 꺼낼지 data에서 꺼낼지 판단

ERC-20 Transfer vs ERC-721 Transfer

둘 다 `Transfer(address,address,uint256)`라서 topic0이 같습니다. ERC-721은 `tokenId`까지 indexed라 `Transfer(indexed address,indexed address,indexed uint256)` 입니다. NFT 가이드 9장의 "topics 4개, data 비어 있음"이 바로 이 차이입니다.

6-3. AMM의 LP Transfer는 왜 안 듣나

AMM도 ERC-20(LP 토큰)이라 `Transfer`를 냅니다. 하지만 LP가 **생기는 곳은 `addLiquidity`, 사라지는 곳은 `removeLiquidity`뿐**이고, 두 이벤트에 `liquidity` 값이 있습니다. 그래서 `LiquidityAdded`에서 더하고 `LiquidityRemoved`에서 빼면 `totalSupply`와 정확히 같아집니다. (LP를 다른 사람에게 보내는 경우까지 "누가 LP를 얼마나 가졌나"를 추적하려면 그때는 `Transfer`도 들어야 합니다.)

CHAPTER 7

startBlock: 어디서부터 읽을지



그림 3. startBlock은 배포 블록보다 앞이면 안전하고, 뒤면 치명적입니다.

startBlock	결과
배포보다 앞 (실제: 138,043,275)	21블록을 더 읽지만 그때는 우리 컨트랙트가 **없었으니** 찾을 이벤트도 없음 → 손해는 약간의 시간뿐
배포보다 뒤 (예: 138,043,300)	`LiquidityAdded(10000, 40000, 20000)`를 놓쳐 Pool이 0/0에서 시작. 다음 swap에서 reserveB −384를 빼면 **음수**. NFT 발행 3건도 놓쳐서 #0~#2가 없는 것으로 기록
0 (지정 안 함)	체인의 처음부터 읽음. 1억 3천만 블록을 훑느라 **매우 느림**

왜 배포 스크립트는 `startBroadcast()` 전에 block.number를 읽나

`uint256 startBlock = block.number;`는 스크립트가 tx를 보내기 **전**에 실행됩니다. 그 시점의 블록 번호는 tx가 실제로 들어갈 블록보다 **항상 앞이거나 같습니다**. 그래서 이 값을 서브그래프의 startBlock으로 쓰면 배포 이벤트를 절대 놓치지 않습니다. 실제로 275(읽은 시점)와 296(실제 배포) 사이에 21블록 차이가 났습니다.

핵심: 서브그래프는 이벤트를 "더하고 빼서" 상태를 만든다

시작점 이벤트 하나만 놓쳐도 이후의 모든 숫자가 틀어집니다. 컨트랙트에 직접 물어보는 `view`와 달리, 서브그래프는 **역사 전체를 봐야만** 현재를 알 수 있습니다.

CHAPTER 8

매핑 코드 (1): AssemblyScript와 생성된 클래스

매핑은 **AssemblyScript**로 씁니다. TypeScript와 문법이 거의 같지만, WebAssembly로 컴파일되기 때문에 몇 가지가 다릅니다.

TypeScript 습관	AssemblyScript (매핑)	이유
`a + b`	**`a.plus(b)`**, `a.minus(b)`	`BigInt`는 객체라 `+` 연산자가 없음
`0`, `1`	**`BigInt.zero()`**, `BigInt.fromI32(1)`	숫자를 BigInt로 바꿔야 함
`a === b` (주소)	**`a.equals(b)`**	Bytes 객체 비교
객체를 바꾸면 끝	**`.save()`** 필수	저장하지 않으면 DB에 반영 안 됨
`undefined`	`null`로 확인 (`if (pool == null)`)	`load`는 없으면 null

8-1. 이벤트 객체에서 꺼낼 수 있는 것

코드	값
`event.params.xxx`	이벤트 인자 (`trader`, `amountIn` …). generated 클래스의 getter
`event.address`	이벤트를 낸 **컨트랙트 주소** (AMM)
`event.block.timestamp`, `event.block.number`	블록 시각, 번호
`event.transaction.hash`	tx 해시
`event.logIndex`	블록 안에서 이 로그의 번호

8-2. 이름 충돌: `Swap as SwapEvent`

```
import { AMM, LiquidityAdded, LiquidityRemoved, Swap as SwapEvent, NFTListed,
NFTListingCancelled, NFTSold }
  from '../generated/AMM/AMM'                          // 이벤트 클래스
import { Pool, Swap, LiquidityEvent, Listing } from '../generated/schema'    // 엔티티 클래스
```

ABI의 이벤트 `Swap`과 스키마의 엔티티 `Swap`이 이름이 같습니다. 한 파일에서 둘 다 쓰려면 하나의 이름을 바꿔야 해서, 이벤트 쪽을 `SwapEvent`로 불러옵니다.

8-3. 엔티티 다루는 3가지 동작

코드	뜻
`Pool.load(id)`	저장된 것 불러오기. 없으면 `null`
`new Pool(id)`	새로 만들기 (아직 저장 전)
`pool.save()`	저장 (새로 만들기 또는 덮어쓰기)

"불러오고, 없으면 만들고, 고치고, 저장"이 모든 상태 엔티티(Pool, Nft)의 공통 패턴입니다. 기록 엔티티(Swap, LiquidityEvent)는 항상 "만들고 저장"만 합니다.

CHAPTER 9

매핑 코드 (2): 핸들러 한 줄씩

9-1. getPool: 처음 한 번만 컨트랙트에 묻기

```
function getPool(address: Address): Pool {
  let pool = Pool.load(address)
  if (pool == null) {
    pool = new Pool(address)
    const amm = AMM.bind(address)       // 매핑 안에서 컨트랙트 view 함수 호출
    pool.tokenA = amm.tokenA()
    pool.tokenB = amm.tokenB()
    pool.reserveA = BigInt.zero()
    pool.reserveB = BigInt.zero()
    pool.totalLiquidity = BigInt.zero()
    pool.swapCount = BigInt.zero()
  }
  return pool
}
```

`Swap` 이벤트에는 `tokenIn` 주소만 있어서, 그게 A인지 B인지 알려면 tokenA 주소가 필요합니다. 그래서 Pool을 **처음 만들 때 한 번만** `AMM.bind(...).tokenA()`로 묻고 저장합니다. 매 swap마다 물으면 이벤트 하나마다 체인 호출이 생겨 인덱싱이 느려집니다. `tokenA`는 `immutable`이라 한 번 물으면 영원히 같습니다.

9-2. handleLiquidityAdded

```
export function handleLiquidityAdded(event: LiquidityAdded): void {
  const pool = getPool(event.address)
  pool.reserveA = pool.reserveA.plus(event.params.amountA)
  pool.reserveB = pool.reserveB.plus(event.params.amountB)
  pool.totalLiquidity = pool.totalLiquidity.plus(event.params.liquidity)
  pool.save()

  const e = new LiquidityEvent(event.transaction.hash.concatI32(event.logIndex.toI32()))
  e.type = 'ADD'
  e.provider = event.params.provider
  e.amountA = event.params.amountA  e.amountB = event.params.amountB
  e.liquidity = event.params.liquidity  e.timestamp = event.block.timestamp
  e.save()
}
```

배포 블록에서	전	후
reserveA	0	10000e18
reserveB	0	40000e18
totalLiquidity	0	20000e18 (= √(10000 × 40000))

`concatI32`는 tx 해시(32바이트) 뒤에 로그 번호를 4바이트로 붙여 유일한 id를 만듭니다. `handleLiquidityRemoved`는 같은 구조에서 `plus` 대신 `minus`, 타입은 `'REMOVE'`입니다.

9-3. handleSwap: 방향에 따라 reserve 적용

```
export function handleSwap(event: SwapEvent): void {
  const pool = getPool(event.address)
  if (event.params.tokenIn.equals(pool.tokenA)) {
    pool.reserveA = pool.reserveA.plus(event.params.amountIn)     // A를 넣고
    pool.reserveB = pool.reserveB.minus(event.params.amountOut)   // B를 받음
  } else {
    pool.reserveB = pool.reserveB.plus(event.params.amountIn)     // B를 넣고
    pool.reserveA = pool.reserveA.minus(event.params.amountOut)   // A를 받음
  }
  pool.swapCount = pool.swapCount.plus(BigInt.fromI32(1))
  pool.save()
  // + Swap 엔티티 1줄 (trader, tokenIn, amountIn, amountOut, timestamp, txHash)
}
```

방향이 바뀌어도 이벤트의 **이름**은 그대로입니다. `amountIn`은 항상 "넣은 양", `amountOut`은 항상 "받은 양"이고, 바뀌는 것은 **어느 reserve에 적용하느냐**뿐입니다.

실제 swap (100 SW42 → R2B)	값
tokenIn	SW42 = pool.tokenA → `if` 블록
reserveA	`10000e18 + 100e18 = 10100000000000000000000`
reserveB	`40000e18 - 38427255620481331088 4 = 3961572744379518668911 6`

9-4. 마켓 핸들러 3개

```
export function handleNFTListed(event: NFTListed): void {
  const l = new Listing(event.params.listingId.toString())
  l.seller = event.params.seller  l.nft = event.params.nft  l.tokenId =
event.params.tokenId
  l.paymentToken = event.params.paymentToken  l.price = event.params.price
  l.status = 'ACTIVE'
  l.createdAt = event.block.timestamp
  l.save()
}
export function handleNFTListingCancelled(event: NFTListingCancelled): void {
  const l = Listing.load(event.params.listingId.toString())
  if (l == null) return
  l.status = 'CANCELLED'
  l.save()
}
export function handleNFTSold(event: NFTSold): void {
  const l = Listing.load(event.params.listingId.toString())
  if (l == null) return
  l.status = 'SOLD'
  l.buyer = event.params.buyer
  l.save()
}
```

컨트랙트의 상태 변화(판매 중 → 판매 완료 / 취소됨)를 그대로 따라 합니다. `if (l == null) return`은 이론상 일어나지 않지만(등록 전에 취소될 수 없음), 혹시 null이면 아무것도 하지 않고 넘어가서 서브그래프가 멈추지 않게 합니다.

9-5. nft.ts: handleTransfer

```
export function handleTransfer(event: Transfer): void {
  const id = event.params.tokenId.toString()
  let nft = Nft.load(id)
  if (nft == null) {
    nft = new Nft(id)          // 처음 보는 번호 = 방금 발행 (from = 0x0)
  }
  nft.owner = event.params.to
  nft.save()
}
```

발행(`from = 0x0`), 판매 등록(나 → AMM), 구매(AMM → 구매자), 취소(AMM → 판매자) 모두 **Transfer 하나로** 처리됩니다. 이 핸들러는 "누가 가졌나"만 신경 쓰고, "왜 옮겨졌나"는 AMM 핸들러가 담당합니다.

CHAPTER 10

빌드와 배포

```
cd code/subgraph
npm install
npx graph codegen            # ABI·스키마 → generated/ 타입
npx graph build              # 매핑을 WebAssembly로 컴파일 → build/
npx graph auth <DEPLOY_KEY>  # Studio의 배포 키 등록 (한 번)
npx graph deploy <slug>      # build/를 IPFS에 올리고 Studio에 알림, 버전 이름을 물어봄 (v0.0.1)
```

단계	하는 일	실패하면
`codegen`	매니페스트 검사(시그니처 ↔ ABI) + 타입 생성	시그니처 오타, 네트워크 불일치 (13장)
`build`	AssemblyScript 컴파일	타입 에러 (`Property '____' does not exist`)
`deploy`	IPFS 업로드 + 인덱싱 시작	키·slug 불일치, 지원 안 하는 네트워크 이름
Studio 동기화	startBlock부터 따라잡기 → "Synced"	핸들러 실행 중 에러 → "Failed" (그 블록에서 멈춤)

로그인 함정: 지갑 네트워크

Studio는 지갑 서명(SIWE)으로 로그인합니다. MetaMask가 **BSC 테스트넷**에 있을 때는 인증 에러가 났고, **Sepolia**로 바꾸자 로그인됐습니다. 로그인 네트워크와 서브그래프가 **읽을** 네트워크(`subgraph.yaml` 의 `chapel`)는 완전히 별개입니다.

고쳐서 다시 배포하기

매핑이나 스키마를 고치면 `codegen → build → deploy` 를 다시 하고, 버전 이름을 `v0.0.2` 처럼 올립니다. 새 버전은 startBlock부터 **다시 인덱싱**합니다. Deploy Key는 비밀번호처럼 다루고, 코드나 git에 넣지 않습니다(`graph auth` 가 내 컴퓨터 설정에만 저장).

CHAPTER 11

검증: 서브그래프 vs 체인

서브그래프는 컨트랙트 상태를 직접 읽지 않고(`tokenA/tokenB`만 예외) 이벤트만으로 장부를 만들었습니다. 그 결과를 `cast call`로 체인에 직접 물어본 값과 비교했습니다.

① 배포 직후

항목	서브그래프	체인 `view`	
reserveA	100000000000000000000	reserveA() = 1e22	✅
reserveB	400000000000000000000	reserveB() = 4e22	✅
totalLiquidity	200000000000000000000	totalSupply() = 2e22	✅
liquidityEvents	1개 ADD, provider = 배포자		✅
nfts	"0", "1", "2" → 배포자	ownerOf(0..2) = 배포자	✅

② NFT #0을 20 SW42에 판매 등록한 뒤

항목	서브그래프	체인	
listings	"0": ACTIVE, price 20e18, seller 배포자, buyer null	listings(0) = (배포자, NFT, 0, 20e18, SW42, true)	✅
Nft "0".owner	AMM	ownerOf(0) = AMM	✅

③ 100 SW42 → R2B swap 뒤

항목	서브그래프	체인	
reserveA	101000000000000000000	101000000000000000000	✅
reserveB	3961572744379518668911 6	3961572744379518668911 6	✅
swapCount	1		✅
swaps	trader 배포자, tokenIn SW42, amountIn 100e18, amountOut 384272556204813310884	anvil 견적과 동일	✅

wei 단위까지 일치

컨트랙트 안에서 일어난 계산을, 서브그래프가 이벤트만 보고 **한 자리도 틀리지 않게** 재현했습니다. 검증된 핸들러는 handleLiquidityAdded, handleTransfer, handleNFTListed, handleSwap입니다. handleLiquidityRemoved, handleNFTSold, handleNFTListingCancelled는 같은 패턴이지만 실제 체인에서는 아직 실행해 보지 않았습니다.

CHAPTER 12

GraphQL 쿼리 문법

스키마의 엔티티 이름이 그대로 쿼리 이름이 됩니다. 단수(`pool(id: …)`)는 하나, 복수(`pools`)는 목록입니다.

```
{
  pools { reserveA reserveB totalLiquidity swapCount }                        # 전부
  listing(id: "0") { status price }                                           # id로 하나
  listings(where: { status: ACTIVE }) { id tokenId price seller }             # 필터
  swaps(first: 10, orderBy: timestamp, orderDirection: desc) { amountIn amountOut }  # 정렬·개수
  nfts(where: { owner: "0x8124b5bee7b8fac709eadd329c9e7c7666289619" }) { id }        # 소문자 주소
}
```

인자	뜻	예
`where`	필터. 칸 이름 + 조건	`{ status: ACTIVE }`, `{ price_gt: "1000" }`, `{ trader_in: [...] }`
`orderBy`, `orderDirection`	정렬	`orderBy: timestamp, orderDirection: desc`
`first`, `skip`	개수와 건너뛰기 (페이지)	`first: 10, skip: 20` = 3페이지

주소는 소문자

서브그래프는 Bytes를 소문자 hex로 저장합니다. 체크섬 대소문자가 섞인 `0x8124B5BE…`로 필터하면 결과가 비어 나옵니다. 프론트에서 쓸 때는 `address.toLowerCase()`를 거치세요.

12-1. 프론트에서 부르는 법 (미리보기)

```
const res = await fetch(QUERY_URL, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ query: '{ listings(where: { status: ACTIVE }) { id tokenId price seller } }' }),
})
const { data } = await res.json()   // data.listings = [...]
```

이 `fetch`를 NFT 메타데이터 때처럼 `useQuery`로 감싸면, 지금의 "listings 전부 읽기"나 "0~19번 스캔"을 쿼리 한 번으로 바꿀 수 있습니다.

CHAPTER 13

우리가 겪은 에러 모음

에러	원인	해결
`Event with signature 'Swap(indexed address,uint256,uint256,uint256)' not present in ABI 'AMM'`	두 번째 인자 `tokenIn`은 `indexed address`인데 `uint256`으로 씀	에러가 보여 주는 "Available events" 목록에서 정확한 시그니처 확인
`Event with signature 'Swap(indexed address,indexed address, uint256,uint256)' …`	쉼표 뒤 **공백** 하나	시그니처에는 공백 없음
`Event with signature 'Transfer(indexed address,indexed address,uint256)' not present in ABI 'TokenizeArt'`	ERC-20 Transfer 시그니처를 씀	ERC-721은 `indexed uint256`
`Conflicting networks used in data sources`	AMM은 `chapel`, TokenizeArt는 `bsc-testnet`	모든 데이터 소스를 `chapel`로
(배포 시) 네트워크 거부	`bsc-testnet`은 The Graph의 네트워크 ID가 아님	`chapel`
`ERROR TS2339: Property '____' does not exist on type 'Swap__Params'`	빈칸이 남아 있음 (build 단계의 타입 검사)	`amountIn` / `amountOut`
`File does not exist: src/amm.ts`	매핑 파일이 아직 없음	매니페스트가 가리키는 파일을 만들어야 codegen 통과
Studio `Authentication error`	지갑이 BSC 테스트넷(97)에 있는 상태로 로그인 서명	Sepolia 등으로 바꾸고 로그인

에러 읽는 요령

graph-cli의 에러는 `Path: dataSources > 0 > eventHandlers > 2`처럼 **매니페스트의 위치**를 알려 줍니다 (0번째 데이터 소스의 2번째 핸들러 = Swap). 그리고 "Available events"처럼 **정답 후보**를 함께 보여 주는 경우가 많습니다.

CHAPTER 14

한계와 주의점

주의점	설명
몇 초 늦음	tx가 채굴되고 인덱서가 그 블록을 처리할 때까지 쿼리 결과가 잠깐 이전 값. 방금 보낸 tx를 확인할 때는 체인 `view`가 더 정확
이벤트가 없는 정보는 못 봄	컨트랙트가 이벤트로 남기지 않은 변화는 인덱싱 불가. 이벤트 설계가 서브그래프의 한계를 정함
핸들러 에러 = 정지	매핑에서 예외가 나면 그 블록에서 "Failed"로 멈춤. 고쳐서 새 버전을 배포해야 함. 그래서 `if (l == null) return` 같은 방어 코드를 둠
체인 재정렬 (reorg)	블록이 뒤집히면 graph-node가 그 블록의 변경을 되돌리고 다시 처리. `prune: auto`는 오래된 과거 상태를 정리하되 재정렬에 필요한 만큼은 보관
Studio 쿼리 주소는 개발용	테스트용이라 사용량 제한이 있음. 서비스용이면 The Graph 네트워크에 퍼블리시하고 API 키를 사용
LP 소유자는 추적 안 함	총량만 추적. "누가 LP를 얼마나"가 필요하면 AMM의 ERC-20 `Transfer`도 들어야 함
체인 하나	chapel만 인덱싱. Sepolia에도 배포하면 같은 코드로 `network: sepolia`인 서브그래프를 하나 더 만들어야 함

CHAPTER 15

스스로 점검하기

Q1. 서브그래프가 하는 일을 한 문장으로? (1장)

컨트랙트 이벤트를 시작 블록부터 다시 읽어, 매핑 규칙대로 정리된 장부를 만들고 GraphQL로 제공합니다.

Q2. view 함수로 답할 수 없는 질문의 예는? (1장)

"지금까지의 swap 기록", "판매 중인 것만", "이 주소가 가진 NFT 번호들".

Q3. 서브그래프를 구성하는 파일 3가지와 역할은? (2장)

schema.graphql(장부 모양), subgraph.yaml(무엇을 어디서부터 볼지), src/*.ts(이벤트 → 장부 변경).

Q4. 데이터 소스가 왜 2개인가? (6장)

NFT의 Transfer는 AMM이 아니라 TokenizeArt 컨트랙트가 내기 때문입니다.

Q5. Swap 엔티티는 immutable, Listing은 아닌 이유는? (5장)

swap은 이미 일어난 기록이라 바뀌지 않고, Listing은 ACTIVE → SOLD/CANCELLED로 상태가 바뀝니다.

Q6. Swap 엔티티의 id에 로그 번호를 붙이는 이유는? (5장)

한 tx에 같은 종류의 이벤트가 여러 개 있을 수 있어서, tx 해시만으로는 id가 겹칠 수 있습니다.

Q7. startBlock이 배포 블록보다 뒤면? 앞이면? (7장)

뒤면 초기 LiquidityAdded와 NFT 발행을 놓쳐 이후 값이 전부 틀어집니다. 앞이면 빈 블록을 더 읽을 뿐이라 시간만 조금 더 듭니다.

Q8. 매니페스트의 이벤트 시그니처에 공백 하나가 있으면? (6장)

topic0(시그니처의 keccak256)이 달라져 ABI에서 찾지 못하고 codegen에서 실패합니다.

Q9. ERC-721 Transfer 시그니처는? ERC-20과 차이는? (6장)

`Transfer(indexed address,indexed address,indexed uint256)`. tokenId가 indexed라는 점이 다르고, topic0은 같습니다.

Q10. AMM의 LP Transfer를 듣지 않고 LP 총량을 아는 방법은? (6-3장)

LP는 addLiquidity에서만 생기고 removeLiquidity에서만 사라지므로, 두 이벤트의 liquidity를 더하고 빼면 됩니다.

Q11. 매핑에서 `a + b` 대신 `a.plus(b)`를 쓰는 이유는? (8장)

graph-ts의 BigInt는 객체라 + 연산자가 없습니다.

Q12. getPool이 tokenA를 매번 묻지 않고 처음 한 번만 묻는 이유는? (9장)

매번 체인 호출을 하면 인덱싱이 느려지고, tokenA는 immutable이라 한 번이면 충분합니다.

Q13. handleSwap의 else 블록에서 amountIn을 reserveB에 더하는 이유는? (9장)

tokenIn이 B라는 뜻이므로, 넣은 양(amountIn)은 B에, 받은 양(amountOut)은 A에서 뺍니다. 이름은 그대로고 적용 대상만 바뀝니다.

Q14. 서브그래프 쿼리에서 주소 필터가 비어 나온다면? (12장)

대소문자가 섞인 주소를 넣었을 가능성. 서브그래프는 소문자로 저장하므로 소문자로 넣어야 합니다.

Q15. Studio 로그인 인증 에러의 원인과 해결은? (10장)

지갑이 BSC 테스트넷에 있는 상태로 로그인 서명을 해서입니다. 이더리움 계열 네트워크(Sepolia)로 바꾸고 로그인했습니다. 인덱싱 네트워크와는 무관합니다.

APPENDIX

명령어와 엔티티 한 장 요약

명령어

명령	하는 일
`npm install`	graph-cli, graph-ts 설치
`npx graph codegen`	매니페스트 검사 + generated/ 타입 생성
`npx graph build`	매핑 컴파일 → build/
`npx graph auth <KEY>`	배포 키 등록
`npx graph deploy <slug>`	IPFS 업로드 + Studio 배포 (버전 이름 입력)

이벤트 → 핸들러 → 엔티티

이벤트	핸들러	엔티티 변화
`LiquidityAdded`	`handleLiquidityAdded`	Pool +, LiquidityEvent(ADD)
`LiquidityRemoved`	`handleLiquidityRemoved`	Pool −, LiquidityEvent(REMOVE)
`Swap`	`handleSwap`	Pool ±, swapCount++, Swap
`NFTListed`	`handleNFTListed`	Listing(ACTIVE)
`NFTListingCancelled`	`handleNFTListingCancelled`	Listing.status = CANCELLED
`NFTSold`	`handleNFTSold`	Listing.status = SOLD, buyer
`Transfer` (NFT)	`handleTransfer`	Nft.owner = to

핵심 숫자

항목	값
네트워크	`chapel` (BSC 테스트넷, chain 97)
AMM	`0x0a1134046cECA785a218A92376fdb542cB9ACA52`
TokenizeArt	`0x7DA2B6BC5330ad801427E6879201b079A774aB15`
startBlock	138,043,275 (배포 tx 블록 138,043,296~297)